

Universidade do Minho

Escola de Engenharia
Departamento de Informática

Biblioteca de Deduplicação a Nível de Bloco sobre FUSE

Relatório do Trabalho Prático de
Tecnologias de Sistemas Operativos

Autores:

Diogo Ribeiro
Nuno Rebelo
Marco Ferreira

Maio de 2026

Conteúdo

1	Introdução	2
2	Arquitectura da biblioteca	2
2.1	Visão geral	2
2.2	Estruturas de dados	3
3	Implementação	4
3.1	Caminho de escrita	4
3.2	Caminho de leitura	5
3.3	Remoção e reciclagem de slots	5
3.4	Persistência	6
3.5	Modelo de concorrência	6
4	Avaliação experimental	7
4.1	Ferramentas desenvolvidas	7
4.2	Metodologia e ambiente	8
4.3	Instrumentação com eBPFs	9
4.4	Benchmarks de Concorrência com FIO	10
4.5	Análise crítica e gargalos identificados	11
5	Conclusão	11

1 Introdução

O presente relatório descreve o desenvolvimento de uma biblioteca de deduplicação para um sistema de ficheiros, realizado no âmbito da unidade curricular de Tecnologias de Sistemas Operativos [6]. O trabalho não se limita à construção da biblioteca em si. O foco recai sobre o processo de a tornar eficiente, sob diferentes cargas de trabalho e em particular sob cargas concorrentes, comparando sucessivas versões da implementação para avaliar o efeito real de cada decisão de desenho.

Os principais desafios encontrados prendem-se com o equilíbrio entre desempenho, correção e simplicidade. Soluções aparentemente diretas revelam consequências subtis quando confrontadas com acessos concorrentes ou com padrões de carga realistas. A maioria das otimizações que fazem sentido em teoria só ficam justificadas após validação empírica. O trabalho procurou, por isso, identificar onde estão os verdadeiros gargalos do sistema e propor alterações cujo benefício fosse mensurável e não meramente conjectural.

A avaliação combina microbenchmarks com instrumentação interna, que contam syscalls e medem a ocupação física do master file, com testes de desempenho conduzidos através da ferramenta FIO, que permite gerar cargas de I/O controladas e reproduzíveis. Cada otimização é medida em comparação direta com a versão anterior, sob diferentes níveis de concorrência e taxas de deduplicação, de modo a tornar visível o seu impacto isolado.

A Secção 2 apresenta a arquitectura da biblioteca, fixando os termos e estruturas usados no resto do documento. A Secção 3 descreve as decisões de desenho dos vários caminhos de operação, identificando a versão inicial e as sucessivas otimizações introduzidas. A Secção 4 apresenta a metodologia experimental, os resultados obtidos e a respetiva análise crítica.

2 Arquitectura da biblioteca

2.1 Visão geral

A biblioteca opera em modo FUSE [1] *passthrough* sobre um ficheiro físico designado *master file*, no qual cada bloco lógico distinto observado pelo sistema é persistido uma única vez. Os ficheiros vistos pelo utilizador passam a ser sequências de referências para posições neste master file, com a duplicação contabilizada num contador de referências por bloco.

A unidade de trabalho é o bloco de 4 KiB e a assinatura de conteúdo de cada bloco é o seu hash SHA-512 [4]. Pedidos não alinhados são rejeitados com EOPNOTSUPP. A Figura 1 resume os principais componentes.

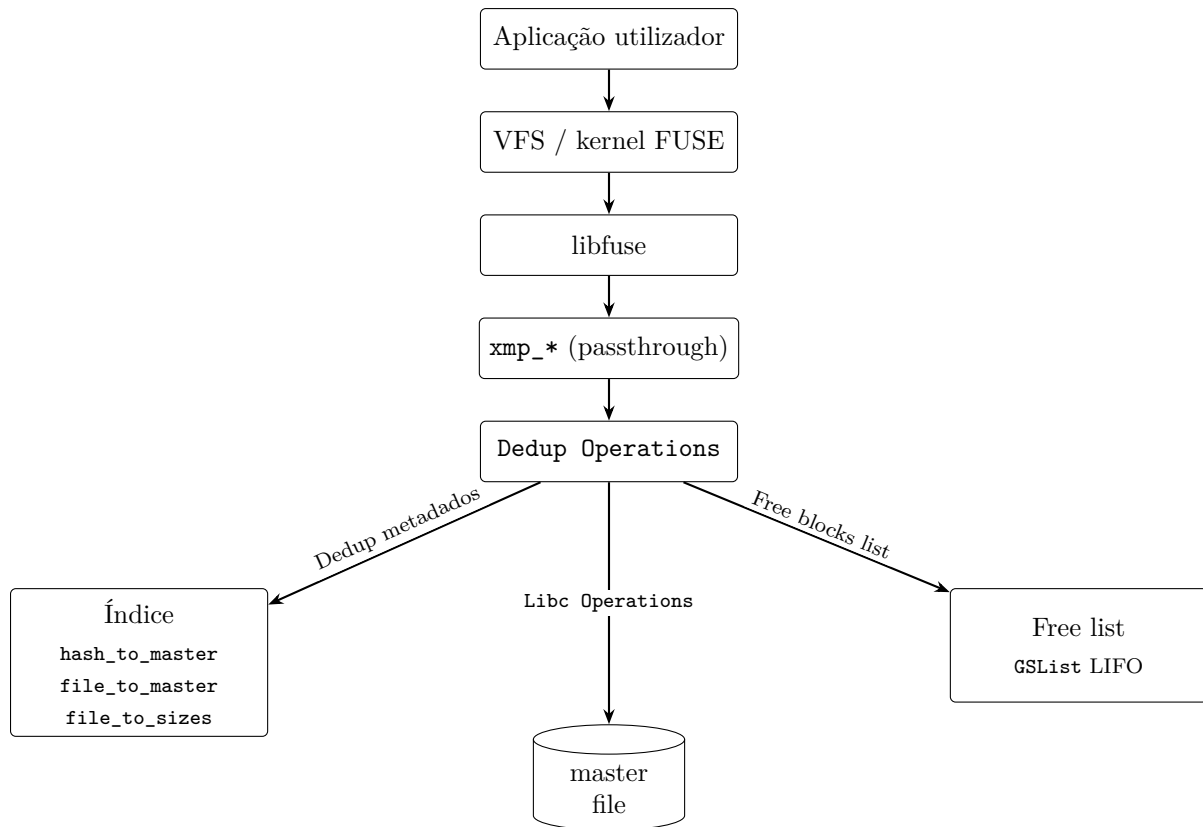


Figura 1: Visão geral da arquitectura. As operações descem pela pilha FUSE até à camada de deduplicação, que consulta as três tabelas de índice e gere a free list de slots libertados. As entradas de `hash_to_master` e `file_to_master` resolvem para uma estrutura `MasterInfo` (Secção 2.2) cujo `masterBlockIndex` determina o offset do I/O alinhado a 4 KiB sobre o master file.

2.2 Estruturas de dados

A informação por bloco físico é mantida numa estrutura `MasterInfo` que guarda o hash SHA-512 do conteúdo, o índice do bloco no master file e um contador de referências, este último atómico para permitir incrementos e decrementos concorrentes sem serialização. A inclusão explícita do hash na estrutura, em vez de o recomputar quando necessário, simplifica o caminho de remoção: ao decrementar o contador para zero, a chave a remover de `hash_to_master` está imediatamente disponível e dispensa uma leitura adicional do bloco do master file só para recalculer o hash.

O índice agrega duas tabelas de dispersão complementares e a lista de slots livres. A primeira tabela, `hash_to_master`, suporta a deteção de duplicados ao mapear cada hash para um ponteiro para o `MasterInfo` correspondente. A segunda, `file_to_master`, indexada pelo par (`path`, `blockIndex`), mapeia cada bloco lógico de cada ficheiro para um ponteiro para o **mesmo** `MasterInfo`. Ambas as tabelas partilham o mesmo objeto em memória, pelo que qualquer atualização ao contador de referências é imediatamente visível através das duas. A free list, mantida como `GSList` [3] encadeada de inteiros, regista os slots no master file que foram libertados por `unlink` ou `truncate` e que podem ser reaproveitados antes de o master file crescer. Uma terceira tabela, `file_to_sizes`, guarda o tamanho lógico de cada ficheiro para satisfazer `stat`.

Toda esta estrutura é coberta por um conjunto coordenado de primitivas de sincronização POSIX [5]: um `pthread_rwlock` sobre os metadados de leitura intensiva, um `pthread_mutex` dedicado à free list e variáveis atómicas para contadores de uso frequente. O detalhe deste modelo é tratado na Secção 3.5.

3 Implementação

3.1 Caminho de escrita

A função `write_dedup` percorreu três gerações sucessivas, motivadas pela mesma preocupação central: garantir correção da deduplicação minimizando o número de syscalls emitidos sob carga.

Na **implementação inicial**, cada bloco lógico do pedido era processado em sequência. Para cada um, calculava-se o hash SHA-512, consultava-se `hash_to_master`, em caso de bloco novo era imediatamente emitido um `pwrite` para a posição correspondente no master file e inserida a entrada nova em `hash_to_master` e em caso de bloco duplicado incrementava-se o contador de referências no `MasterInfo` respectivo. No final as novas entradas do `file_to_master` eram inseridas para cada bloco e o size do file atualizado no `file_to_sizes`. Ou seja, para um pedido FUSE de 64 KiB com todos os blocos distintos, esta implementação resultava em 16 syscalls de escrita disparados em sequência, ainda que os slots alocados fossem fisicamente contíguos.

A **implementação com batching** divide o processamento em fases. Uma primeira fase decide o destino de cada bloco do pedido sem emitir I/O: percorre os blocos, calcula os hashes e marca cada bloco como HIT (já existe no índice) ou MISS (novo, com um `MasterInfo` pendente). Uma segunda fase aloca os slots no master file para os MISSes, drenando a free list antes de fazer append. A terceira fase emite os syscalls de escrita agrupando MISSes em *runs* cujos índices físicos são sequenciais: cada run é descarregada num único `pwrite`. Para um pedido com 16 MISSes consecutivos basta uma única chamada de sistema, em vez de 16.

A **implementação com três passes e correção em caso de falha** acrescenta a esta estrutura uma fase final de consolidação. As atualizações dos índices `hash_to_master` e `file_to_master` são adiadas até depois do `pwrite`. Se este falhar a meio do flush, uma rotina de *rollback* devolve os slots alocados à free list e liberta os `MasterInfo` pendentes, sem deixar entradas no índice oficial a apontar para conteúdo que nunca chegou ao disco. Esta separação entre decisão, flush e consolidação é fundamental para suportar o controlo de concorrência refinado descrito na Secção 3.5, em que cada passe pode usar um nível de lock distinto. A Figura 2 sumariza estas fases.

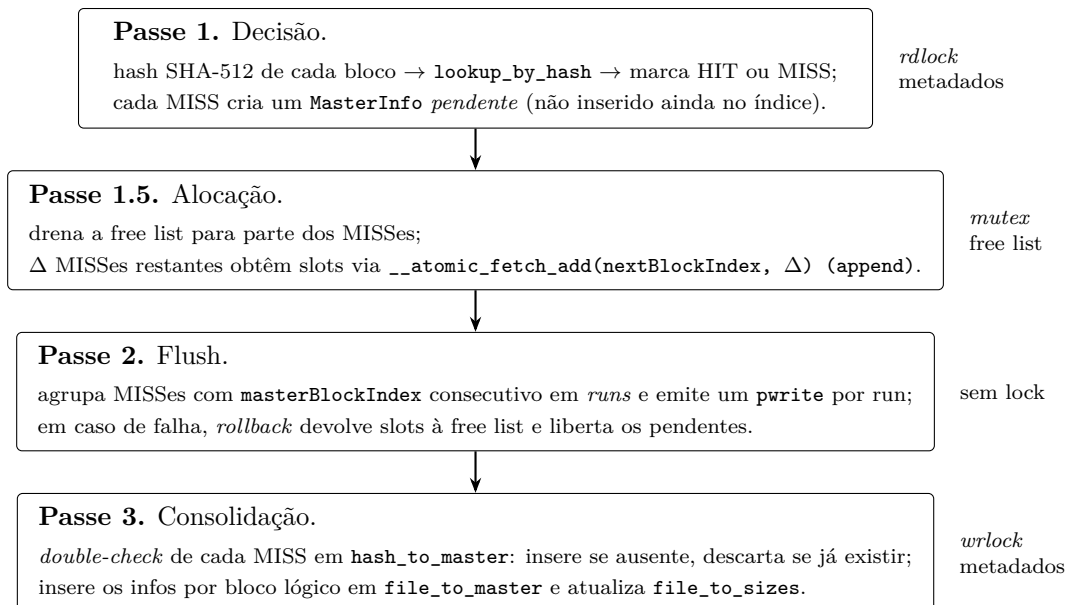


Figura 2: Fluxo da função `write_dedup` em três passes, com os substeps internos de cada um. Cada passe usa o nível mínimo de sincronização que a sua semântica exige, libertando a fase de I/O (Passe 2) de qualquer lock.

3.2 Caminho de leitura

A função `read_dedup` implementa leitura com proteção de concorrência, evoluindo através de três gerações.

Na **implementação inicial**, cada bloco lógico solicitado resultava num *pread* separado ao arquivo mestre, mesmo que múltiplos blocos mapeassem para endereços físicos consecutivos. Ler N blocos implicava N syscalls de leitura, gerando overhead significativo apesar da correção lógica.

A **implementação com batching** explora o facto de blocos lógicos contíguos poderem corresponder a posições físicas próximas, mesmo quando dispersos pela deduplicação. A estratégia consiste em obter os índices físicos de todos os blocos do pedido com *lookups* em `file_to_master`, ordená-los (*insertion sort* para $N \leq 32$, *qsort* acima) e identificar nesse vector os grupos de índices fisicamente consecutivos. Cada grupo é então lido com um único *pread* para um buffer temporário, do qual se copiam os blocos para as suas posições lógicas no buffer de saída. O número de syscalls cai de N para $\approx N/(\text{tamanho médio do grupo})$, com ganhos significativos sempre que há reutilização.

A **implementação com controlo de concorrência refinado** reduz o escopo do lock crítico. Em vez de manter o mutex durante a ordenação e leitura (que são operações locais à thread), protege-se apenas o lookup dos metadados com um read-write lock (`pthread_rwlock_rdlock` no início da fase 1 do lookup). Este read-lock é libertado imediatamente após copiar os índices mestres para o array local, permitindo que múltiplas threads façam leituras concorrentes sem bloquearem-se. As fases 2–5 (ordenação, agrupamento, leitura física e cópia) executam-se sem lock, sendo completamente thread-safe por operarem em estruturas locais e fazerem *preads* a offsets disjuntos. A Figura 3 sumariza este fluxo.

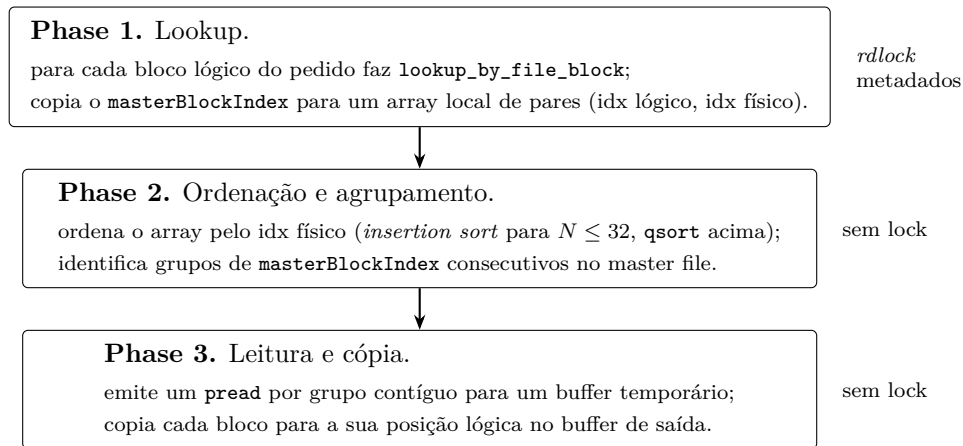


Figura 3: Fluxo da função `read_dedup`. Só a Phase 1 toma o `rdlock` dos metadados, durante o tempo estritamente necessário para copiar os `masterBlockIndex` relevantes para uma estrutura local. O restante trabalho corre fora de qualquer lock.

3.3 Remoção e reciclagem de slots

As operações de `unlink` e `truncate` libertam blocos lógicos que deixam de ter referências e, eventualmente, slots no master file que ficam livres para reuso. Uma versão ingénua trataria cada bloco isoladamente, decrementando o contador de referências do `MasterInfo` respetivo e, quando este chegasse a zero, removeria a entrada do índice e adicionaria o slot à free list. Para um ficheiro com N blocos, esta abordagem implica N pares de aquisições de locks distintos, custo que cresce linearmente com o tamanho do ficheiro.

A **implementação em lote** agrupa todas as remoções de um pedido numa única rotina, `remove_blocks_dedup_batch`, adquirindo um único *write lock* sobre os metadados. Dentro

da secção crítica percorrem-se todos os blocos lógicos do ficheiro a remover, consultando o mapeamento `file_to_master` para obter o respetivo `MasterInfo`. Cada entrada lógica (`path`, `blockIndex`) é removida deste índice, e a partir do `MasterInfo` obtêm-se a `hash` do bloco e o seu `refcount`. Caso esta seja a última referência lógica ao bloco (isto é, quando `refcount = 1`), a entrada correspondente é também removida da tabela `hash_to_master` em tempo $O(1)$, e o slot físico é acumulado numa lista local. Só no final é adquirido o lock da free list para concatenar todos os slots de uma vez.

Esta otimização reduz o número de aquisições de lock de $2N$ para 2, independentemente do tamanho do ficheiro, tornando o custo da remoção proporcional ao trabalho útil e não às operações de sincronização.

A free list que recebe os slots libertados é mantida como `GSList` encadeada em política LIFO, com inserção e remoção em $O(1)$ na cabeça. Uma versão exploratória usou uma árvore de *extents* com fusão automática de intervalos contíguos, mas foi descartada após medições terem mostrado que o caminho quente é dominado por escritas em *append*, onde o ganho de *coalescing* era marginal e não compensava o custo adicional por operação.

3.4 Persistência

Os metadados da deduplicação são persistidos em disco no final da execução. A **implementação inicial** gravava cada `MasterInfo` duas vezes, uma por cada tabela de dispersão em que aparecia. A **implementação com pares indexados** eliminou essa duplicação ao gravar os valores uma única vez num ficheiro partilhado e ao deixar cada tabela guardar apenas índices que apontam para esses valores, recorrendo às funções `ghash_save_indexed_pair` e `ghash_load_indexed_pair` para escrever e reconstruir as tabelas no arranque.

A **implementação com batching de persistência** agrupa múltiplas entradas num buffer de 10 MB e descarrega-o num único `pwrite`, reduzindo drasticamente o número de syscalls de escrita. A durabilidade é reforçada pela flag `O_SYNC` no `open` dos ficheiros de metadados, forçando sincronização com disco e garantindo *crash consistency* parcial.

A API de *persistência* foi mantida genérica para reutilização entre estruturas distintas. *Encoders* mais especializados poupariam espaço em disco mas acoplariam a persistência às estruturas concretas, dificultando evoluções futuras. Este compromisso favorece a evolução do sistema sobre a otimização pontual de espaço em metadados.

3.5 Modelo de concorrência

O sistema FUSE, em modo *multi-thread* por defeito, despacha em paralelo os pedidos para as funções de operação. Garantir correção sob esta concorrência exigiu evoluir o esquema de sincronização desde um único mutex global, presente nas duas primeiras gerações da biblioteca, até um conjunto granular de primitivas afinadas para o perfil de acesso de cada estrutura. A Tabela 1 sumariza a correspondência final entre estruturas e mecanismos de sincronização.

Tabela 1: Primitivas de sincronização por estrutura.

Estrutura	Primitiva
<code>hash_to_master</code> , <code>file_to_master</code> , <code>file_to_sizes</code>	<code>pthread_rwlock</code>
<code>MasterInfo.refcount</code>	<code>_Atomic uint32_t</code>
<code>free_block_list</code>	<code>pthread_mutex</code> dedicado
<code>nextBlockIndex</code> (apontador de <i>append</i>)	<code>_Atomic uint64_t</code>

A propriedade central deste esquema é que as secções críticas cobrem apenas a reserva dos offsets no master file e as inserções no índice, não a operação de I/O. Uma vez reservado o offset, seja por `__atomic_fetch_add` sobre `nextBlockIndex` ou por pop da free list, o `pwrite`

ou **pread** correspondente executa fora de qualquer lock. Múltiplas threads servem assim I/O em paralelo sobre offsets disjuntos do master file.

O *read-write lock* sobre os metadados é favorecido em relação a um mutex porque a operação dominante sob carga real é a leitura, em particular a consulta de **file_to_master** no caminho de leitura e os lookups de hash no Passe 1 do caminho de escrita. Múltiplas threads podem executar estas consultas em simultâneo. Apenas a consolidação do Passe 3 e a remoção em lote tomam o lock em modo exclusivo. Para evitar deadlocks adota-se uma hierarquia estrita: qualquer fluxo que precise de adquirir tanto o lock dos metadados como o mutex da free list fá-lo sempre por esta ordem, nunca o contrário.

A separação do lock de metadados em relação ao lock da free list elimina contenção desnecessária entre operações de natureza diferente: uma leitura concorrente não bloqueia uma libertação de slots e duas remoções em lote conseguem entrelaçar a fase de varrimento de metadados com a fase de devolução de slots à free list. As variáveis atômicas reduzem ainda mais a serialização. O contador **nextBlockIndex** permite que múltiplos writes em append reservem ranges contíguos de slots via **__atomic_fetch_add**. O contador de referências de cada **MasterInfo** é incrementado em paralelo sem precisar de lock exclusivo.

Falta resolver a possibilidade de duas threads inserirem em simultâneo o mesmo conteúdo no índice de hashes. A janela existe apenas entre o lookup do Passe 1 e o insert do Passe 3 e a sua materialização exige que duas threads escrevam o mesmo bloco em microssegundos. Mesmo assim, para evitar duplicados, o Passe 3 efetua um *double-check*: antes de inserir o **MasterInfo** pendente, faz novo lookup sob o *write lock*. Se já existe entrada, descarta o seu **MasterInfo** pendente, devolve o slot alocado à free list e usa o **MasterInfo** já presente, incrementando-lhe o contador de referências. O custo desta defesa é um **pwrite** ocasionalmente desperdiçado para um slot que é imediatamente reciclado. A garantia é a ausência de duplicados no índice mesmo sob disputa entre threads.

4 Avaliação experimental

A avaliação combina benchmarks baseados em eBPF que medem diretamente o número de syscalls, a utilização da page cache e a ocupação de armazenamento, com benchmarks de I/O sintético gerados pela ferramenta FIO. Os primeiros permitem isolar o efeito de cada otimização sobre o caminho quente da biblioteca. Os segundos avaliam o comportamento global sob diferentes níveis de concorrência.

4.1 Ferramentas desenvolvidas

Para suportar a avaliação experimental da biblioteca, foram desenvolvidas 3 principais ferramentas:

- **programa eBPF cachestat** - mede a utilização da page cache
- **programa eBPF syscounter** - conta o número de syscalls efetuadas e mede o tempo de execução de cada uma
- **script Python** - gera IO com padrões simples de duplicação

O programa cachestat é uma adaptação do programa com o mesmo nome disponibilizado no repositório oficial do bcc. Em relação ao programa original, foram incluídos um filtro por PID e um output em formato JSON, de maneira a integrar o programa no pipeline de benchmark que será explicado mais abaixo.

O syscounter constitui uma adaptação do programa desenvolvido nas aulas que conta chamadas ao sistema. Em relação à versão original, mantêm-se o filtro por PID, os tracepoints apenas de syscalls relacionadas ao sistema de ficheiros e também o filtro que só deixa passar descritores

de ficheiros. Foi ainda adicionada a funcionalidade de medição do tempo de execução de cada syscall. Para realizar esta medição, regista-se o timestamp, em nanossegundos, do momento em que a chamada foi feita, ou seja, no tracepoint de entrada; posteriormente, no tracepoint de saída, obtém-se outro timestamp, fazendo-se depois a subtração para obter o tempo de execução. Por fim, foi também adicionado um output em formato CSV, para que, tal como no cachestat, se possa facilmente integrar esta ferramenta no pipeline.

Finalmente, foi concebido um script em Python que simula uma workload de duplicação. Embora seja possível gerar este tipo de cargas com ferramentas como o FIO, optou-se por desenvolver este pequeno script de maneira a podermos misturar diferentes operações em simultâneo que o FIO não permite facilmente (e.g., unlinks). Além disso, desta forma torna-se mais fácil respeitar as limitações impostas para o projeto (append-only) e garantir controlo total sobre a carga gerada, o que é relevante para fazer medições tão granulares como a contagem de syscalls e o tempo de execução em nanossegundos. Sendo assim, é possível configurar o workload usando uma configuração com os seguintes campos:

- **write_pcnt** - proporção de requests que serão writes.
- **read_pcnt** - proporção de requests que serão reads.
- **unlink_pcnt** - proporção de requests que serão unlinks.
- **dup_pcnt** - proporção de blocos que serão escritos da pool de duplicados.
- **block_range** - intervalo para o número de blocos de cada request.
- **num_ops** - número de requests a serem feitos
- **num_files** - número de ficheiros diferentes a usar
- **num_unique_dup_blocks** - tamanho da pool de blocos duplicados

A configuração usada para os benchmarks que se apresentam na secção 4.2 foi: 49, 49, 2, 15 ou 75, [1,30], 200000, 1000 e 200, para a ordem apresentada acima.

Para juntar todas estas ferramentas, foi elaborado um pipeline (na forma de um script bash) que limpa todos os dados do FUSE; arranca este último; lança um syscounter com o PID do passthrough; lança outro syscounter com o PID do workload; lança um cachestat com o PID do passthrough; executa a workload; quando esta última acaba, os eBPFs são interrompidos, fazendo cada um o seu output; destrói-se o passthrough. No final, é usado um outro script Python que recolhe os resultados produzidos por cada um dos eBPFs, calculando médias e desvios padrão.

4.2 Metodologia e ambiente

Todas as medições foram feitas na mesma máquina, com 10 núcleos lógicos e o ficheiro *master* alojado num sistema de ficheiros local. Cada configuração foi executada 10 vezes. Os valores apresentados nas tabelas e gráficos correspondem à média dessas 10 execuções, sendo o desvio-padrão calculado para confirmar que não existem variações inesperadas.

Os benchmarks de I/O foram conduzidos com a ferramenta FIO [2] (*Flexible I/O Tester*). A configuração usada foi: 5 GB no total, **bssplit** com 4 KB (60%), 16 KB (25%) e 64 KB (15%), percentagem de duplicação configurável, vários níveis de concorrência (1, 2, 4, 8, 10 e 12 jobs) e **psync** como engine.

Uma dificuldade inicial surgiu na geração de blocos únicos em paralelo: blocos que deviam ser distintos acabavam por coincidir entre processos FIO diferentes, fazendo com que a percentagem de duplicados observada superasse a configurada. A solução foi atribuir a cada job uma sequência aleatória própria, usando **randrepeat=1** e um **randseed** diferente por processo, garantindo que cada job gera o seu conjunto de blocos de forma independente e a deduplicação observada passa a corresponder ao comportamento esperado dentro de cada job.

4.3 Instrumentação com eBPFs

Esta primeira bateria de testes compara três versões do sistema FUSE (*passthrough* básico, *dedup v1* e *dedup com batching*) em cenários de 15% e 75% de duplicados, sob carga não concorrente. A utilização de disco é calculada após terminar o *passthrough*, ou seja, além de considerar o tamanho da *master file*, considera-se também o espaço usado pelos metadados persistidos. A versão *passthrough* (sem deduplicação) serve de referência para percebermos o impacto das funcionalidades adicionadas. A versão *dedup v1* corresponde à implementação inicial, com um *syscall* por bloco. A versão *dedup com batching* corresponde à segunda versão, que agrupa *MISSes* contíguos tanto em escritas como em leituras, diminuindo assim o número de acessos a disco. Esta segunda versão foi concebida porque, ao analisar a execução com recurso ao *perf*, foi possível perceber que a função onde são feitas as escritas passou 32% do tempo a calcular o hash e 30% a executar a *syscall* *pwrite*. Sendo assim, fazer *batching* de *syscalls* foi considerada uma otimização relevante a se fazer. A Tabela2 sumariza os resultados.

Tabela 2: Benchmark: tempo total, ocupação física (DU) e contagem e tempo médio das *syscalls* *pwrite* e *pread* observadas no nível do FUSE.

Versão	Dups	Tempo (s)	DU (MB)	pwrite		pread	
				N	μs	N	μs
Passthrough	15%	56,8	1438	98 387	58,9	93 053	17,2
Passthrough	75%	57,1	1467	97 844	59,6	93 370	17,5
Dedup v1	15%	93,4	1349	1 299 107	4,4	1 412 265	2,4
Dedup v1	75%	83,9	412	378 941	5,9	1 418 174	2,2
Dedup batching	15%	91,5	1349	783 470	5,8	492 256	5,9
Dedup batching	75%	81,6	412	343 155	5,6	1 219 560	2,4

Tabela 3: Benchmark: estatísticas de utilização da *page cache* do programa *passthrough*.

Versão	Dups	PC hits	PC <i>dirtyies</i>	PC total (MB)
Passthrough	15%	122 963	1 525 559	1781
Passthrough	75%	142 715	1 511 279	1810
Dedup v1	15%	298 742	1 296 697	1612
Dedup v1	75%	1 208 913	377 948	711
Dedup batching	15%	288 642	1 296 296	1624
Dedup batching	75%	1 169 219	377 804	720

Tabela 4: Benchmark: tempo médio por chamada observada no programa de utilizador (a *syscall* observada é *write* para escritas e *pread* para leituras).

Versão	Dups	<i>write</i> (μs)	<i>pread</i> (μs)
Passthrough	15%	272,7	214,4
Passthrough	75%	275,4	217,0
Dedup v1	15%	536,6	237,0
Dedup v1	75%	467,4	226,3
Dedup batching	15%	521,5	231,9
Dedup batching	75%	453,3	217,7

O efeito mais visível é a redução de ocupação física que a deduplicação introduz: com 75% de duplicados o *master file* + metadados passam de 1467 MB (sem deduplicação) para 412 MB, uma redução de cerca de 72%, próxima do máximo teórico para esta taxa de duplicação. Com

15% de duplicados o ganho de espaço é mais modesto, como seria de esperar, mas continua a corresponder a uma redução de aproximadamente 6%.

Em compensação, a versão *dedup v1* introduz um número significativamente maior de syscalls do que o *passthrough*, com mais de 1,3 milhões de `pwrite` no cenário de 15% de duplicados, contra os cerca de 98 mil do *passthrough*. Este custo é o sintoma direto do loop bloco-a-bloco da implementação inicial: cada bloco de 4 KiB origina o seu próprio `pwrite`, ao passo que o *passthrough* herda os tamanhos definidos pelo `bssplit` do FIO. A versão com batching reduz este número para 783 mil, cerca de 40% menos no cenário de 15% de duplicados. No cenário de 75% de duplicados a redução é menor (de 379 mil para 343 mil, cerca de 10%) porque a maioria dos blocos é HIT e nunca chega a gerar uma escrita, deixando menos espaço para batching. No caminho de leitura o ganho do batching é bastante mais acentuado no cenário de 15% de duplicados, em que o número de `pread` cai de 1,41 milhões para 492 mil.

Os tempos médios por chamada (Tabela 4) confirmam as diferenças entre os dois sistemas FUSE. Do ponto de vista do utilizador, a média do `write` sobe de cerca de $273\ \mu\text{s}$ no *passthrough* para $537\ \mu\text{s}$ em *dedup v1* a 15% de duplicados, sintoma direto do trabalho extra de hashing e gestão de índice imposto a cada chamada. O batching atenua ligeiramente este valor, deixando-o em $521\ \mu\text{s}$.

Por fim, ao nível da *page cache*, o cenário a 75% de duplicados mantém-se coerente com a utilização de disco, pois as páginas marcadas como *dirty* caem cerca de quatro vezes face ao *passthrough*, refletindo a poupança real em escrita para disco, enquanto as *hits* aumentam quase uma ordem de grandeza pela pressão de leitura sobre o índice persistido.

4.4 Benchmarks de Concorrência com FIO

Os testes avaliaram a evolução das versões *dedup sem concorrência* e *dedup com concorrência* com o aumento do número de jobs.

A Figura 4 compara a evolução dos IOPS e da *bandwidth* em função do número de *jobs* para as duas versões da biblioteca de deduplicação.

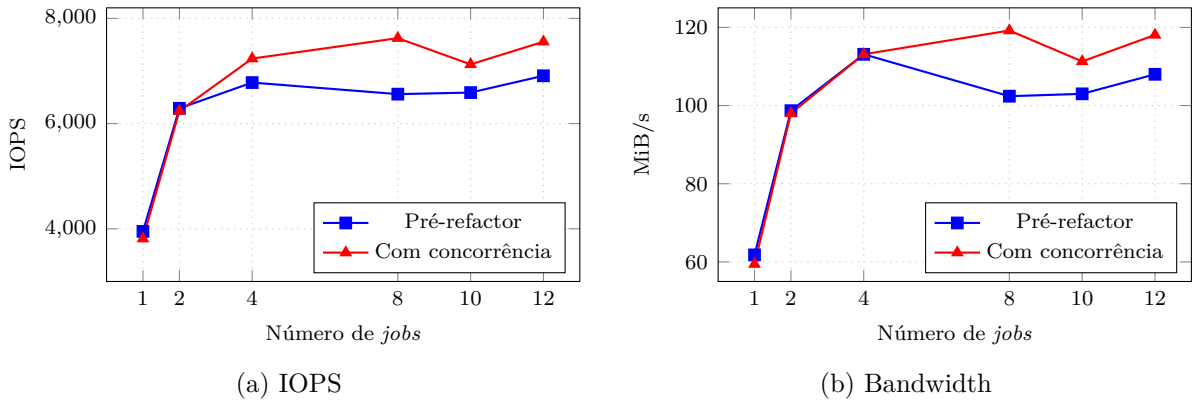


Figura 4: Escalonamento de IOPS e *bandwidth* com o número de *jobs*, sob carga FIO com 50% de duplicados.

Na versão *pré-refactor*, os IOPS sobem de 3952 para 6780 entre 1 e 4 *jobs*, acompanhados por uma subida da *bandwidth* de 61,8 para 113,1 MiB/s. A partir desse ponto, a evolução entra em *plateau*, com pequenas oscilações entre 8, 10 e 12 *jobs*, sugerindo contenção causada pelo mutex global que serializa toda a atividade sobre as estruturas partilhadas.

Na versão *com concorrência*, o crescimento mantém-se mais consistente: passa de 3815 IOPS com 1 *job* para 7236 com 4 *jobs*, atingindo 7624 com 8 *jobs* e 119,2 MiB/s de *bandwidth*. Em praticamente todos os níveis acima de 1 *job*, esta versão supera a anterior, indicando menor contenção e uma utilização mais eficiente do paralelismo disponível. O ganho relativo no pico (8

jobs) é de aproximadamente 16%, validando empiricamente que o refactor traduz o paralelismo permitido pelas estruturas granulares em ganho real de *throughput*.

A partir de 8 *jobs*, ambas as versões aproximam-se do limite do *hardware* de teste, que dispõe de 10 núcleos. As oscilações observadas entre 10 e 12 *jobs* são consistentes com efeitos típicos de carga saturada (escalonamento do *kernel*, espera em *locks* e interação com a *page cache*) e não traduzem uma degradação estritamente linear.

4.5 Análise crítica e gargalos identificados

A leitura conjunta dos três conjuntos de medições permite identificar três gargalos restantes. O primeiro é o custo do hash SHA-512 no caminho de escrita, que continua a ser computado em série dentro de cada bloco. Mesmo com I/O fora dos *locks* e múltiplas *threads* a calcular *hashes* em paralelo, o tempo de `pwrite` médio mantém-se na ordem dos microssegundos, deixando o SHA-512 como uma componente significativa do tempo total nas escritas pequenas. No sentido de melhorar ainda mais a performance do sistema desenvolvido, cogitou-se a implementação de assincronismo entre o cálculo das *hashes* e as chamadas a `pwrite`. Este mecanismo poderia ser implementado tanto com uma biblioteca de I/O assíncrono (e.g., `io_uring`), possibilitando o cálculo de *hashes* enquanto se realizam escritas, ou, de forma simétrica, através da utilização de uma *thread pool* responsável por computar os *hashes*, o que permitiria realizar as escritas em paralelo. Teoricamente, ambas as abordagens são semelhantes, sendo necessária a sua implementação para determinar o verdadeiro impacto na performance.

O segundo gargalo é a contenção residual sobre o *write lock* do Passe 3 da função `write_dedup`. Sob carga concorrente intensa, várias *threads* aguardam pela aquisição exclusiva do lock para consolidar os respetivos *batches* no índice. A natureza curta desta fase mitiga o impacto, mas não o anula. Reduzir ainda mais o tempo desta fase exigiria fragmentar o índice por *shards*, uma evolução adiada para trabalho futuro.

O terceiro gargalo é a ausência de *coalescing* na free list. O *workload* configurado no FIO realiza essencialmente operações de *append* e não exercita cenários onde o espaço previamente libertado é reutilizado de forma agressiva. Em workloads dinâmicos com muitas remoções intercaladas com escritas, a free list LIFO simples pode fragmentar-se e degradar o batching de *runs* no Passe 2.

5 Conclusão

Este trabalho descreve uma biblioteca de deduplicação sobre FUSE, desenvolvida ao longo de três versões sucessivas. Cada iteração isolou uma decisão de desenho distinta. A primeira foi a introdução do *batching* de syscalls. A segunda foi a separação do caminho de escrita em três passes com consolidação atrasada, libertando a fase de I/O de qualquer lock. A terceira foi o refinamento do controlo de concorrência através de um *rwlock* sobre os metadados, um mutex dedicado à free list e contadores atômicos para os campos de uso frequente. Os resultados experimentais mostram efeito mensurável em cada uma: o *batching* reduz em cerca de 40% o número de `pwrite` emitidos no caminho de escrita no cenário de 15% de duplicados (Tabela 2) e o refinamento da sincronização traduz-se num ganho de cerca de 16% no IOPS de pico sob carga concorrente face à versão com mutex global.

A principal limitação conhecida é a ausência de *coalescing* na free list, que pode degradar o *batching* em workloads de remoção intensiva. No perfil de carga *append-only* alvo deste trabalho não há fragmentação acumulada da free list, mas em workloads dinâmicos esta limitação passaria a ser relevante.

O trabalho futuro tem várias direções identificadas. A mais promissora face às medições é pipelinar o cálculo de SHA-512 com o I/O, sobrepondo o hashing dos blocos seguintes com o `pwrite` dos já assinados, atacando diretamente o gargalo identificado na Secção 4.5. Outra

direção é substituir o `path` por um inode numérico nas chaves de `file_to_master`: o `perf` mostra que a comparação e hashing de strings de path dominam o tempo de lookup em hierarquias profundas, sem benefício prático em manter a chave legível. A migração para a interface *low-level* do FUSE permitiria controlo mais fino sobre a fila de pedidos e eliminaria parte do trabalho de tradução feito pela API *high-level*. Vale ainda notar que para o futuro a persistência poderia singularizar o hash que serve simultaneamente de chave em `hash_to_master` e de campo em `MasterInfo`, eliminando a duplicação de escrita do mesmo valor nos dois contextos.

Referências

- [1] libfuse contributors, *libfuse: Filesystem in Userspace reference implementation*. <https://github.com/libfuse/libfuse>, 2024.
- [2] J. Axboe, *FIO: Flexible I/O Tester*. <https://github.com/axboe/fio>, 2024.
- [3] The GNOME Project, *GLib Reference Manual*. <https://docs.gtk.org/glib/>, 2024.
- [4] National Institute of Standards and Technology, *Secure Hash Standard (SHS)*, FIPS PUB 180-4. U.S. Department of Commerce, 2015.
- [5] IEEE and The Open Group, *IEEE Standard for Information Technology: Portable Operating System Interface (POSIX) Base Specifications, Issue 7*, IEEE Std 1003.1-2017, 2018.
- [6] Departamento de Informática, Universidade do Minho, *Tecnologias de Sistemas Operativos: Enunciado do Trabalho Prático*, 2026.
- [7] B. K. R. Vangoor, V. Tarasov, and E. Zadok, *To FUSE or Not to FUSE: Performance of User-Space File Systems*. In Proceedings of the 15th USENIX Conference on File and Storage Technologies (FAST), pp. 59–72, 2017.